UNIT I

Introduction

1.1 Embedded Systems:

An embedded system is a computational system that is developed based on the integration of both hardware and software in order to perform a given task. It can be said that a dedicated computer system has been developed for some particular reason. But it is not our traditional computer system or general-purpose computers; these are the Embedded systems that may work independently or attached to a larger system to work on a few specific functions. These embedded systems can work without human intervention or with little human intervention.

Components of Embedded Systems

1. Hardware 2. Software 3. Firmware

1.1.1 Examples of Embedded Systems

- Digital watches
- Washing Machine
- Toys
- Televisions
- Digital phones
- Laser Printer
- Cameras
- Industrial machines
- Electronic Calculators
- Automobiles
- Medical Equipment

1.1.2 Application of Embedded System

- Home appliances
- Transportation
- Health care
- Business sector & offices
- Defense sector
- Aerospace
- Agricultural Sector

1.1.3 Characteristics of an Embedded System

- **Performs specific task**: Embedded systems perform some specific function or tasks.

- **Low Cost**: The price of an embedded system is not so expensive.
- **Time Specific**: It performs the tasks within a certain time frame.
- **Low Power**: Embedded Systems don't require much power to operate.
- **High Efficiency**: The efficiency level of embedded systems is so high.
- **Minimal User interface**: These systems require less user interface and are easy to use.
- **Less Human intervention**: Embedded systems require no human intervention or very less human intervention.
- **Highly Stable**: Embedded systems do not change frequently mostly fixed maintaining stability.
- **High Reliability**: Embedded systems are reliable they perform tasks consistently well.
- **Use microprocessors or microcontrollers**: Embedded systems use microprocessors or microcontrollers to design and use limited memory.
- **Manufacturable**: The majority of embedded systems are compact and affordable to manufacture. They are based on the size and low complexity of the hardware.

1.1.4 Block Structure of Embedded System

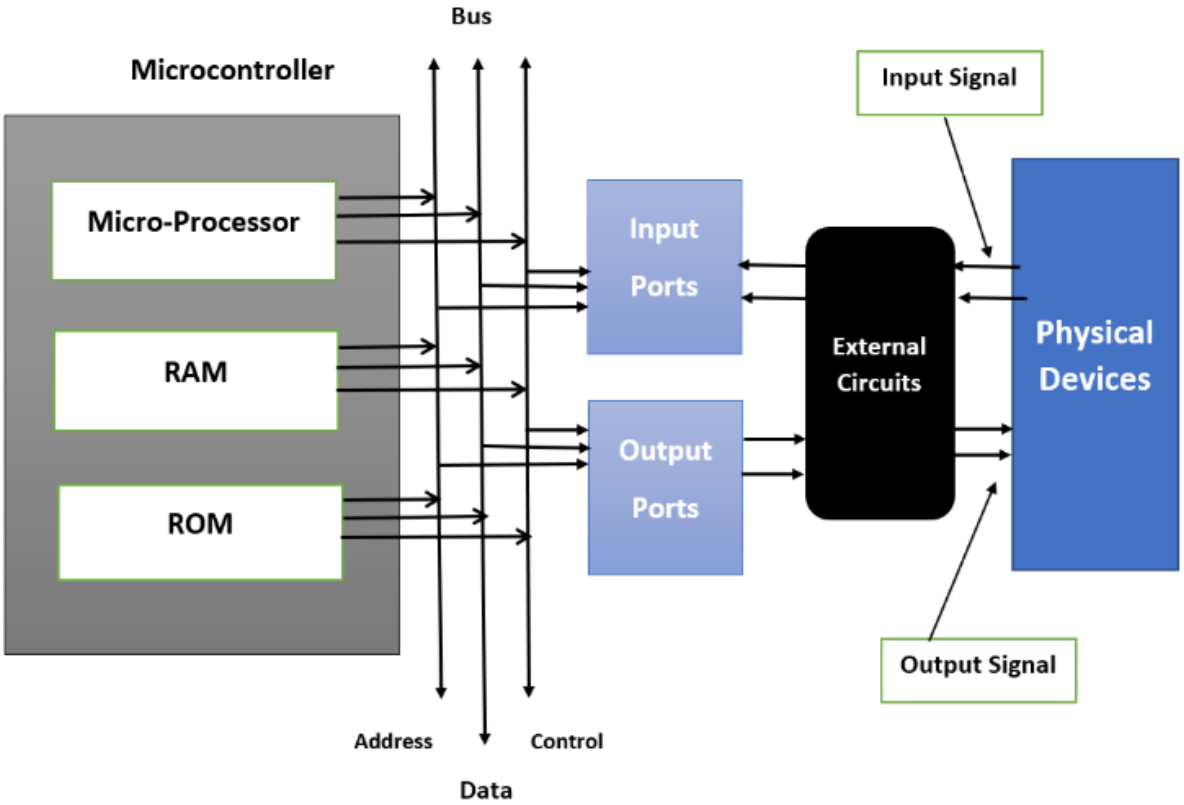


Figure 1 Block Structure of Embedded System

The block structure of an embedded system integrates hardware and software to perform dedicated tasks, featuring a central processing unit (CPU/microcontroller), memory (ROM/RAM), input

sensors, output actuators, communication interfaces, and a power supply. It processes real-world signals using ADCs, acts on data via controllers, and runs firmware, often with a Real-Time Operating System (RTOS).

Core Blocks of an Embedded System:

- **Processor (CPU/Microcontroller):** The brain of the system that fetches, decodes, and executes instructions (e.g., Microprocessor, Microcontroller, DSP, ASIC).
- **Memory:** Stores code and data. ROM (Flash/EEPROM) holds the program code (firmware), while RAM is used for temporary data storage during execution.
- **Input & Output (I/O) Interfaces:** Connects the processor to the external world.
    - **Input Devices:** Sensors, switches, and keyboards capture physical signals.
    - **Output Devices:** Actuators, LEDs, and LCDs display results or perform actions.
- **Converters:**
    - **Analog-to-Digital Converter (ADC):** Converts analog sensor signals into digital data.
    - **Digital-to-Analog Converter (DAC):** Converts digital CPU output into analog signals for actuators.
- **System Components:**
    - **Power Supply:** Provides the necessary DC power (often 5V or 3.3V).
    - **Clock/Oscillator:** Generates timing signals to synchronize operations.
    - **Reset Circuit:** Resets the processor to a known state.
- **Software/Firmware:** The application code is programmed into ROM that directs the hardware to perform specific functions.

1.1.5 Advantages of Embedded System

- Small size.
- Enhanced real-time performance.
- Easily customizable for a specific application.

1.1.6 Disadvantages of Embedded System

- High development cost.
- Time-consuming design process.
- As it is application-specific less market available.

1.2 Real time systems and Real time scheduling

1.2.1 Real-Time Systems

Definition: Systems where correctness depends on both logical results and the time at which results are produced.

Types:

Hard Real-Time: Missing a deadline is a total system failure (e.g., airbag controllers, pacemakers).

Soft Real-Time: Missing deadlines is undesirable but not fatal (e.g., video streaming).

Characteristics: Predictability, dependability, and event-driven operation.

1.2.2 Real time scheduling

Scheduling ensures tasks are executed based on urgency.

- **Static Scheduling (Off-line):** Decisions are made at design time based on known task parameters, reducing runtime overhead.
- **Dynamic Scheduling (On-line):** Decisions are made at runtime, allowing for greater flexibility.
- **Key Algorithms:**
    - **Rate Monotonic (RM):** A static priority algorithm that assigns higher priorities to tasks with shorter periods.
    - **Earliest Deadline First (EDF):** A dynamic algorithm that prioritizes tasks with the closest deadlines.
    - **First-In-First-Out (FIFO):** Processes tasks in the order they arrive.
    - **Round-Robin:** Assigns equal time slots to tasks.

1.3 Processor basics and System-On-Chip

1.3.1 Processor basics

- **Central Processing Unit (CPU):** The main processor, managing tasks, instructions, and data processing.
- **Memory:** Includes on-chip RAM, ROM, and cache.
- **Graphics Processing Unit (GPU):** Handles visual, graphic, and image processing tasks.

- **Interconnects:** Buses and network-on-chip (NoC) structures that transfer data between components.
- **Peripherals & Connectivity:** Modules for Wi-Fi, Bluetooth, USB, and cellular radios.
- **Analog/Mixed-Signal Blocks:** ADCs, DACs, and other components for signal processing.

1.3.2 System-On-Chip

A System-on-Chip (SoC) is an integrated circuit that consolidates all computer components—CPU, memory, GPU, and I/O interfaces—onto a single silicon chip. SoCs enhance efficiency and reduce size/power consumption compared to traditional multi-chip systems, commonly used in smartphones, IoT devices, and modern computing.

Key Advantages of SoCs

- **Miniaturization:** Compact size enables portable, smaller device design.
- **Efficiency:** Lower power consumption and reduced cost.
- **Performance:** Higher speed due to shorter, on-chip communication paths

Common Applications

- **Mobile Devices:** Smartphones and tablets.
- **Embedded Systems:** IoT, automotive, and smart appliances.
- **Computing:** Laptops, servers, and, in some cases, supercomputers.

1.4 Arduino

Arduino is an open-source electronics platform combining easy-to-use hardware (microcontroller boards) and software (IDE) for building interactive projects. It simplifies programming by enabling sensors and motors to interact with the real world. Synonyms include microcontroller board, DIY electronics platform, or maker board.

Key Aspects & Usage Examples:

Purpose: Ideal for students, artists, and hobbyists to create interactive objects or environments.

Examples: Automating lights, building robots, monitoring temperature, IoT applications (using WiFi/Bluetooth), and controlling motors

.

Components: Consists of physical programmable circuit boards (e.g., Arduino Uno) and the Arduino Software (IDE) to write/upload code.

Functionality: Uses input/output pins to read data from sensors (e.g., light, temperature) and control outputs (e.g., LED lights, motors).

1.5 ARM processor and its Features

ARM (Advanced RISC Machine) processors are a family of CPUs based on the RISC (Reduced Instruction Set Computing) architecture. Originally developed by Acorn Computers in the 1980s, ARM processors are now designed by Arm Holdings and widely licensed by other companies (like Apple, Qualcomm, Samsung, etc.) for use in their chips.

It is a widely-used computer chip known for its efficiency and versatility. Designed by ARM Limited using a streamlined RISC architecture these processors are licensed to various companies rather than manufactured directly.

ARM unique business model allows tech companies to customize and build processors for diverse devices, from smartphones and tablets to computers and smart devices. Their exceptional balance of processing power and energy efficiency has made them the preferred choice for mobile computing, enabling longer battery life without compromising performance.

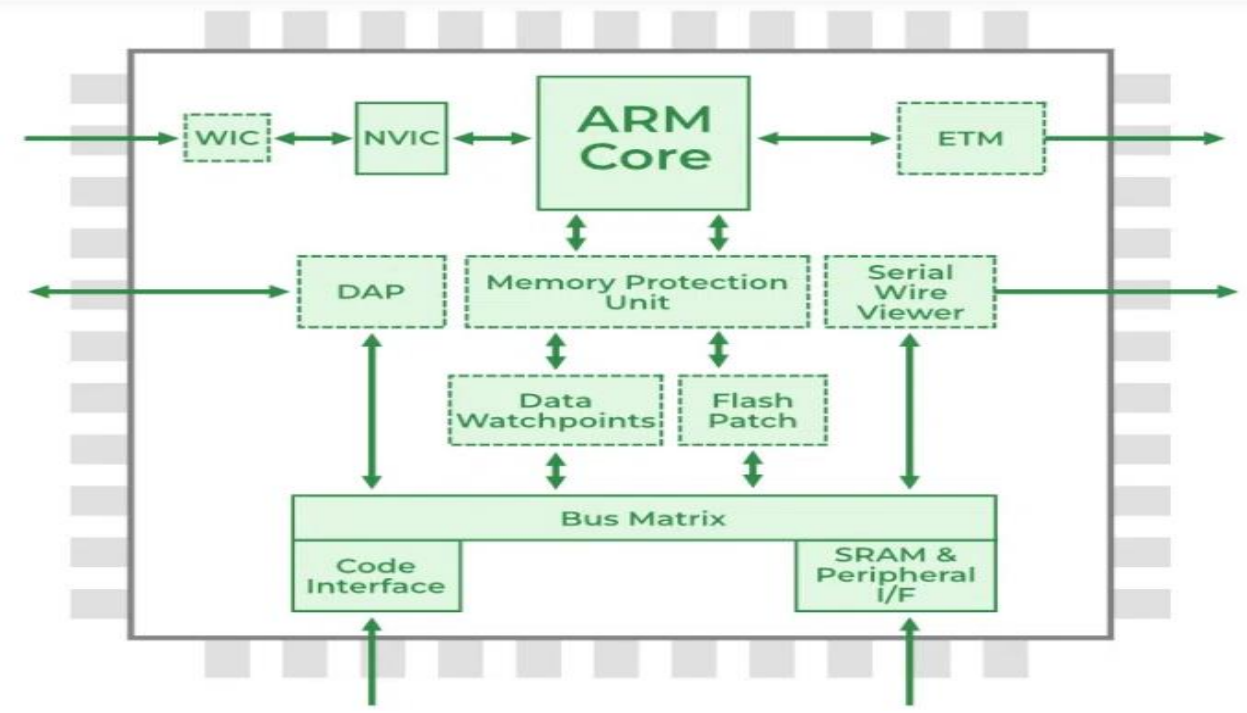


Figure 2. Advanced RISC Machine (ARM)

Common ARM Processor Families

Cortex-M Series : For microcontrollers (low power, real-time control)

Cortex-R Series : For real-time systems (e.g., automotive, robotics)

Cortex-A Series : For application processors (e.g., smartphones, tablets)

Neoverse : For infrastructure and cloud computing

Apple Silicon (e.g., M1, M2) : Custom ARM-based processors for Macs

1.5.1 Features of ARM Processor

Multiprocessing Systems

ARM processors are designed for use in multiprocessing systems, where more than one processor is utilized to process information concurrently. The first Asymmetric Multiprocessing (AMP) processor, introduced under the name ARMv6K, supported up to four CPUs with integrated hardware support.

Modern ARM processors are available in single-core to octa-core (or more) configurations and are commonly used in multi-core SoCs (System on Chips) to enable efficient multitasking and enhanced performance.

Tightly Coupled Memory

The memory of ARM processors is tightly coupled. This has a very fast response time. It has low latency (quick response) that can also be used in cases of cache memory being unpredictable. TCM provides deterministic access times, making it ideal for real-time and safety-critical applications.

Memory Management

ARM processors include advanced memory management components such as the Memory Management Unit (MMU) and the Memory Protection Unit (MPU). These systems are essential for:

- Efficient memory utilization
- Virtual memory implementation (MMU)
- Protecting critical sections of memory (MPU)
- Enabling operating system support (e.g., Linux)

Thumb-2 Technology

Thumb-2 Technology, introduced in 2003, enables variable-length instruction sets. It extends the original 16-bit Thumb instruction set to include 32-bit instructions, improving code density and execution performance. This dual-width capability offers:

- Reduced memory usage.
- Better performance than standard 16-bit Thumb.
- Improved compatibility with existing ARM instructions.

One-Cycle Execution Time

ARM processor is optimized for each instruction on the CPU. Each instruction is of a fixed length that allows time for fetching future instructions before executing the present instructions. ARM has CPI (Clock Per Instruction) of one cycle.

Pipelining

Processing of instructions is done in parallel using pipelines. Instructions are broken down and decoded in one pipeline stage. The channel advances one step at a time to increase throughput (rate of processing).

A large number of Registers

ARM processors provide a large set of general-purpose registers to minimize memory access operations. These registers store data, addresses, and control information, functioning as fast, local storage for the CPU. This architecture:

- Reduces memory bottlenecks.
- Speeds up execution.
- Improves overall system efficiency.

Reasons ARM Architecture is Valuable

Given below are the reasons which makes the ARM processor valuable to the us :

- **Widespread Adoption Across Devices**: ARM (Advanced RISC Machine) architecture is one of the most commonly used electronic architectures in the world. It is widely adopted in smartphones, feature phones, laptops, and embedded systems.
- **Better Alternative to x86 in Many Areas**: While x86 processors dominate the server market with high performance, ARM offers cost-effective, energy-efficient, and smaller-sized processors. It is increasingly seen as a better fit for portable and scalable applications due to these advantages.
- **Low Power Consumption and Better Battery Life**: ARM processors require less power to operate, making them ideal for battery-powered devices. This leads to longer battery life, which is critical for mobile and portable technologies.
- **Compact and Cost-Effective Design**: ARM processors are smaller in size, helping reduce device form factors. Their simplified RISC-based design leads to lower production costs, making them more affordable for large-scale use.

Definition and Overview

Definition: IIoT refers to the network of intelligent, automated, and interconnected machinery, devices, and sensors used to collect, exchange, and analyze data to optimize industrial operations.
Core Objective: To transform vast amounts of industrial data into actionable insights for real-time decision-making, predictive maintenance, and operational efficiency.
IIoT vs. Consumer IoT: While IoT often focuses on consumer convenience (smart home), IIoT deals with high-stakes industrial applications requiring extreme reliability, security, and low latency.

- **Used in High-Performance Computing**: ARM is not limited to mobile devices it powers Fugaku, the world's fastest supercomputer (as of 2021). This shows ARM's ability to scale for high-performance computing (HPC).
- **Flexibility for Hardware Designers**: ARM provides more design feasibility to hardware engineers. Designers have the ability to customize processor cores and maintain better control over the supply chain, unlike fixed-architecture solutions like x86.

1.6 Definition and Overview

- **Definition:** IIoT refers to the network of intelligent, automated, and interconnected machinery, devices, and sensors used to collect, exchange, and analyze data to optimize industrial operations.
- **Core Objective:** To transform vast amounts of industrial data into actionable insights for real-time decision-making, predictive maintenance, and operational efficiency.
- **IIoT vs. Consumer IoT:** While IoT often focuses on consumer convenience (smart home), IIoT deals with high-stakes industrial applications requiring extreme reliability, security, and low latency.

1.6.1 History

- **1968:** Invention of the Programmable Logic Controller (PLC) by Dick Morley, initiating automated assembly lines.
- **1980s-90s:** Adoption of M2M (machine-to-machine) communication and SCADA (supervisory control and data acquisition) systems.
- **1999:** Kevin Ashton coins the term "Internet of Things".

- **2002-2006:** Rise of cloud computing and the development of the OPC Unified Architecture (OPC UA) protocol for secure, interoperable machine communication.
- **2010s-Present:** The emergence of Industry 4.0, 5G, and Artificial Intelligence (AI) enabled the modern IIoT era.

1.6.2 Physical Design (Hardware Components)

The physical design describes the tangible, "things" aspect of the system.

- **Sensors:** Gather raw data (temperature, pressure, vibration, humidity, gas).
- **Actuators:** Execute commands, such as stopping a machine or adjusting valves.
- **IIoT Gateways:** Connect field devices (OT) to the internet/cloud (IT), enabling protocol translation, local data filtering, and edge computing.
- **Computing Nodes:** Embedded systems, microcontrollers (Arduino/Raspberry Pi), or edge servers that process data locally.
- **Connectivity Modules:** Wi-Fi, Ethernet, Bluetooth, and cellular (4G/5G) modems.

1.6.3 Logical Design (Software and Data Flow)

Logical design dictates how components work together, data flows, and functions are performed.

Watsoo

- **IoT Functional Blocks:** Device (sensing/actuation), Communication (data transmission), Services (monitoring/discovery), Management (governance), Security (authentication/integrity), and Application (interface).
- **Communication Models:**
    - **Publish-Subscribe:** Devices (Publishers) send data to a broker (e.g., MQTT), which delivers it to consumers (Subscribers). Efficient for many devices.
    - **Request-Response:** Clients make requests to servers.
- **Key Protocols:**
    - **MQTT (Message Queuing Telemetry Transport):** Lightweight, ideal for low-power sensors.
    - **OPC UA:** Standardized, secure protocol for industrial interoperability.

- **CoAP (Constrained Application Protocol):** For resource-constrained device

1.6.4 Levels (IIoT Architecture)

IIoT systems are typically structured in a multi-layered, modular architecture.

1. **Device Layer:** Sensors, machines, and actuators (OT).
2. **Network Layer:** Connectivity components (Wi-Fi, 5G, LoRaWAN).
3. **Data Processing/Edge Layer:** Local data processing to reduce latency.
4. **Platform/Cloud Layer:** Centralized analysis, storage, and visualization.
5. **Application Layer:** Software applications (ERP/MES) that utilize the data for decision-making

1.6.5 Communication Technologies

- **Wireless:** Wi-Fi, Bluetooth Low Energy (BLE), Zigbee, LoRaWAN, Cellular (4G LTE/5G).
- **Wired:** Ethernet, CAN bus, Serial (UART).
- **5G:** Crucial for low-latency, real-time control of robots and mobile machines.

1.6.6 Applications

- **Predictive Maintenance:** Analyzing machine data to detect potential failure before it occurs, reducing downtime.
- **Smart Factory/Manufacturing:** Automation, quality control, and robotic arm coordination.
- **Supply Chain & Logistics:** Real-time tracking of assets, inventory management, and autonomous transport.
- **Energy Management:** Monitoring and optimizing power usage, managing smart grids.
- **Oil and Gas:** Monitoring offshore pipelines for leaks and controlling extraction processes.
- **Agriculture:** Soil moisture monitoring, precision farming, and livestock tracking